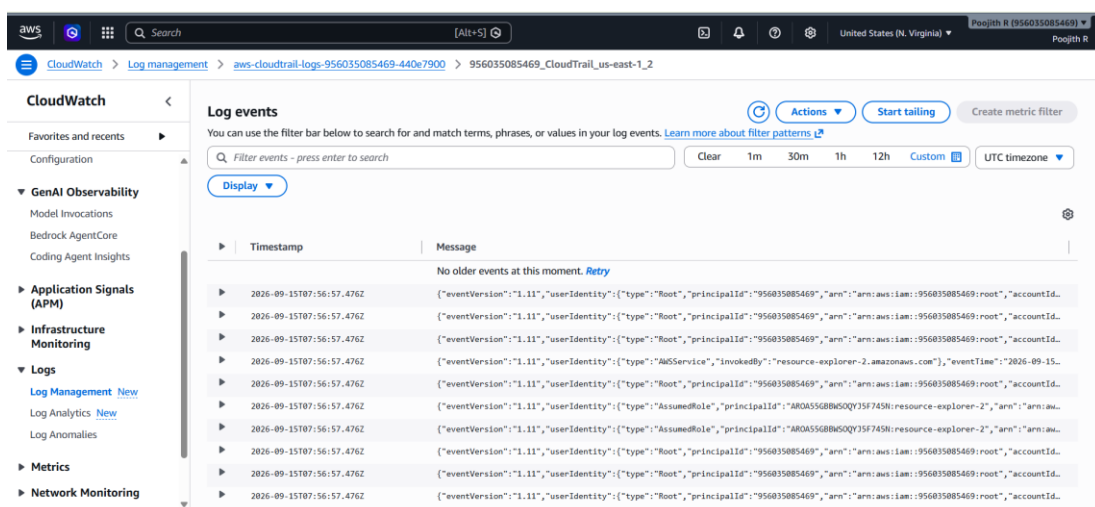
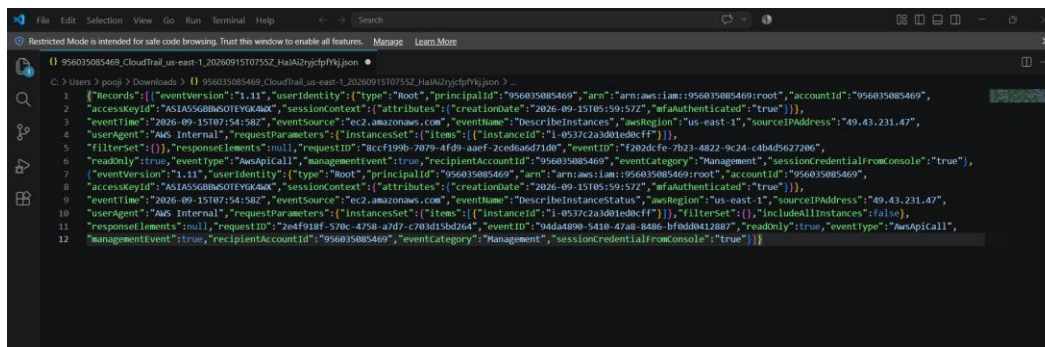
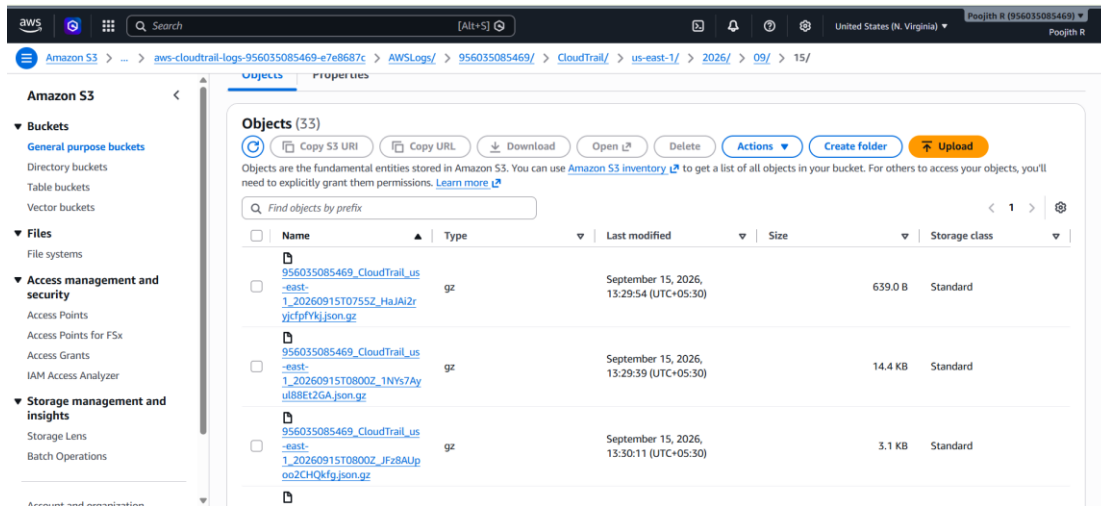






2. Enable SNS for CloudTrail to send alerts via email.

Implementation

1. Opened **Amazon SNS**.
2. Created/configured an SNS topic for CloudTrail notifications.
3. Created an **email subscription** for the SNS topic.
4. Sent the subscription confirmation request.
5. Opened the confirmation email.
6. Confirmed the SNS subscription.
7. Verified that the subscription status changed to **Confirmed**.
8. Configured the CloudTrail/CloudWatch notification workflow to publish alerts to the SNS topic.
9. Generated/tested a CloudTrail event.
10. Verified that the notification was delivered through SNS.

aws [Search] [Alt+S] United States (N. Virginia) Poojith R (956035085469) Poojith R

Amazon SNS > Topics > Cloudtrail_SNS

Amazon SNS

- Dashboard
- Topics
- Subscriptions
- ▼ Mobile
 - Push notifications
 - Text messaging (SMS)

Cloudtrail_SNS [Edit] [Delete] [Publish message]

Details

Name Cloudtrail_SNS	ARN arn:aws:sns:us-east-1:956035085469:Cloudtrail_SNS	Display name -	Type Standard
Topic owner 956035085469			

Subscriptions (0) [Edit] [Delete] [Request confirmation] [Confirm subscription] [Create subscription]

[Search]

ID	Endpoint	Status	Protocol
No subscriptions found			

Amazon SNS > Subscriptions > Create subscription

New Feature
Amazon SNS now supports High Throughput FIFO topics. [Learn more](#)

Topic ARN
arn:aws:sns:us-east-1:956035085469:Cloudtrail_SNS

Protocol
The type of endpoint to subscribe
Email

Endpoint
An email address that can receive notifications from Amazon SNS.
ravutla.10@gmail.com

After your subscription is created, you must confirm it. [Info](#)

► **Subscription filter policy - optional** [Info](#)
This policy filters the messages that a subscriber receives.

► **Redrive policy (dead-letter queue) - optional** [Info](#)
Send undeliverable messages to a dead-letter queue.

Gmail Search mail Upgrade

Compose

- Inbox
- Starred
- Snoozed
- Sent
- Drafts
- Spam
- Purchases
- More

Labels +

AWS Notification - Subscription Confirmation [Inbox x]

AWS Notifications <no-reply@sns.amazonaws.com> to me 2:34 PM (0 minutes ago) ☆ ☺ ↶ ⋮

You have chosen to subscribe to the topic:
arn:aws:sns:us-east-1:956035085469:Cloudtrail_SNS

To confirm this subscription, click or visit the link below (if this was in error no action is necessary):
[Confirm subscription](#)

Please do not reply directly to this email. If you wish to remove yourself from receiving all future SNS subscription confirmation requests please send an email to [sns-out-out](#)

[Reply] [Forward] [More]

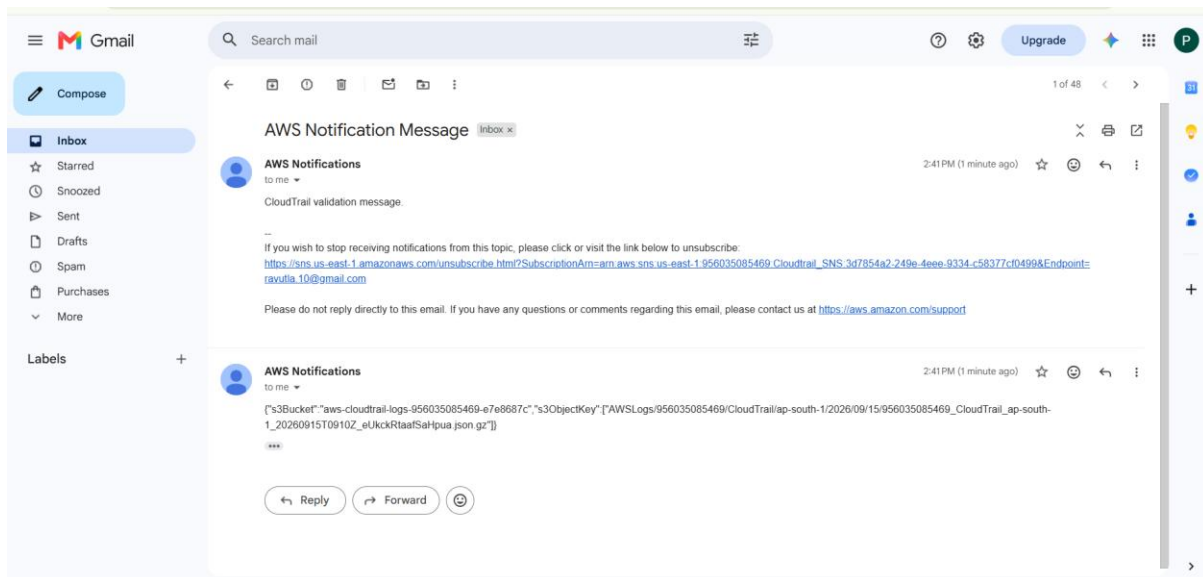
sns.us-east-1.amazonaws.com/confirmation.html?TopicArn=arn:aws:sns:us-east-1:956035085469:Cloudtrail_SNS&Token=2336412f37fb69

aws Simple Notification Service

Subscription confirmed!
You have successfully subscribed.

Your subscription's id is:
arn:aws:sns:us-east-1:956035085469:Cloudtrail_SNS:3d7854a2-249e-4eee-9334-c58377c9b499

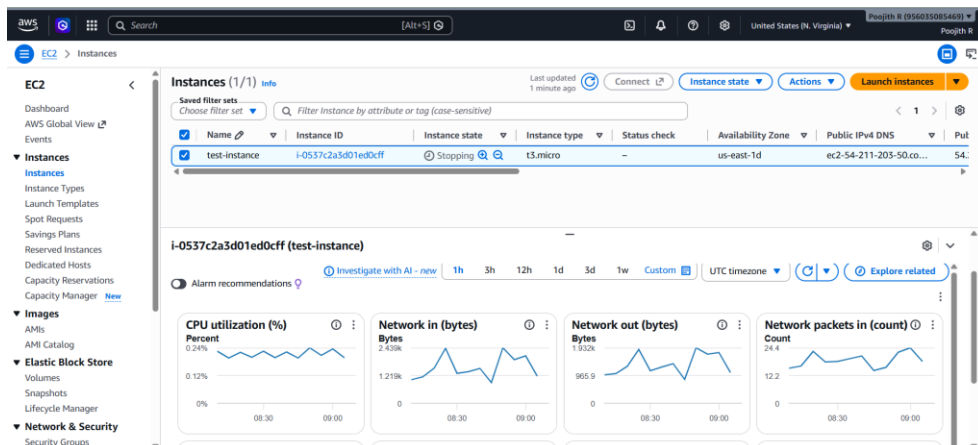
If it was not your intention to subscribe, [click here to unsubscribe](#).

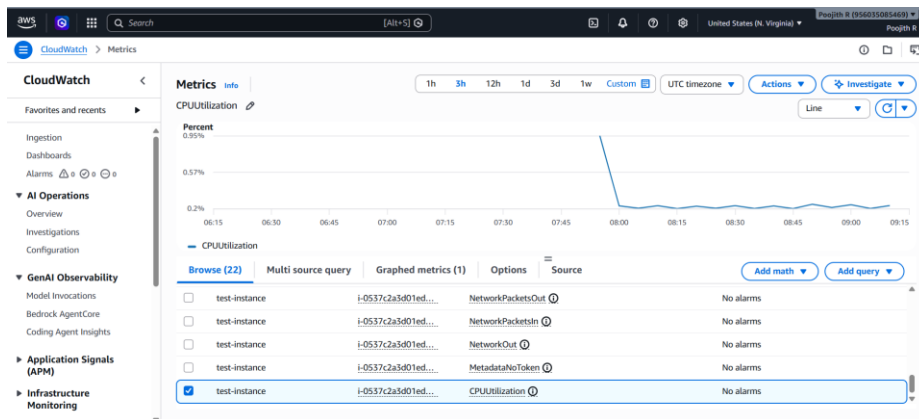


3. Configure CloudWatch monitoring and record the CPU utilization and other metrics of EC2.

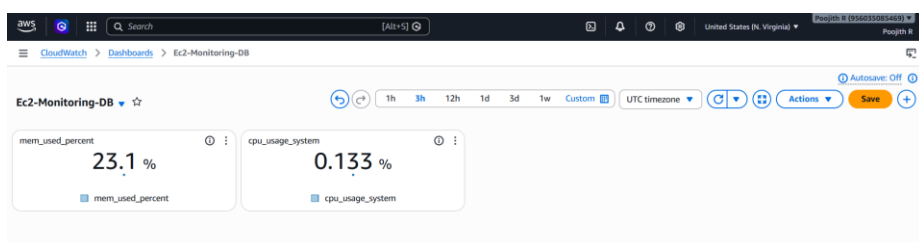
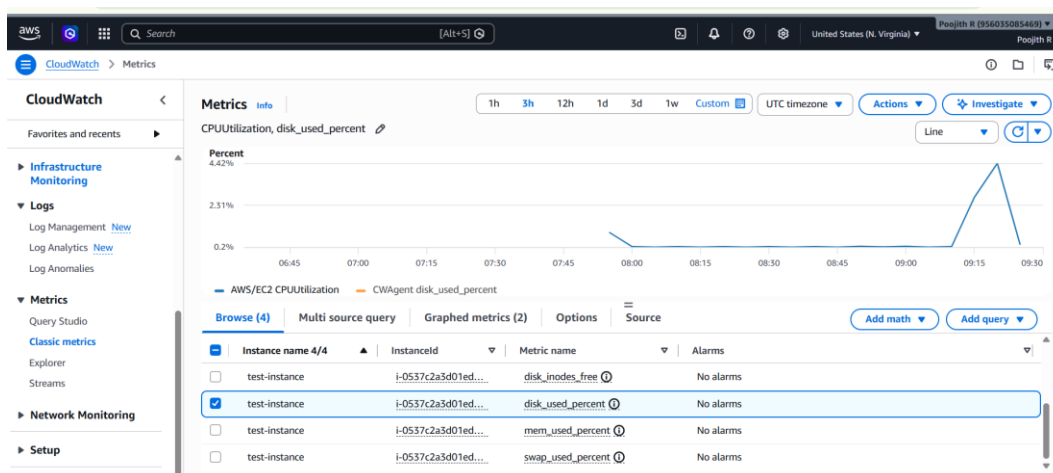
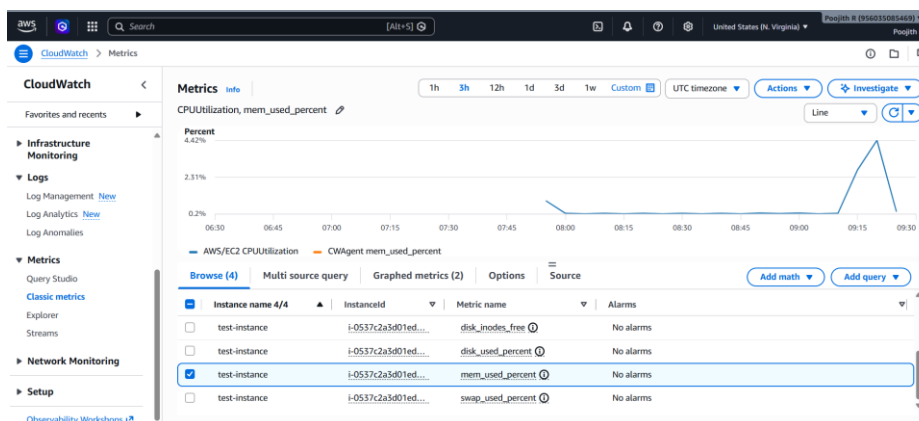
Implementation

1. Opened **Amazon CloudWatch**.
2. Navigated to **Metrics** → **EC2**.
3. Selected the required EC2 instance.
4. Monitored the default EC2 metrics.
5. Configured monitoring for:
 - CPU Utilization
 - Network metrics
 - Disk-related metrics where available
 - Status check metrics
6. Verified the EC2 CPU utilization graph.
7. Selected an appropriate monitoring period for the metrics.
8. Created/configured a CloudWatch dashboard for EC2 monitoring.
9. Added the required EC2 metrics to the dashboard.
10. Verified that the metrics were being updated from the EC2 instance.





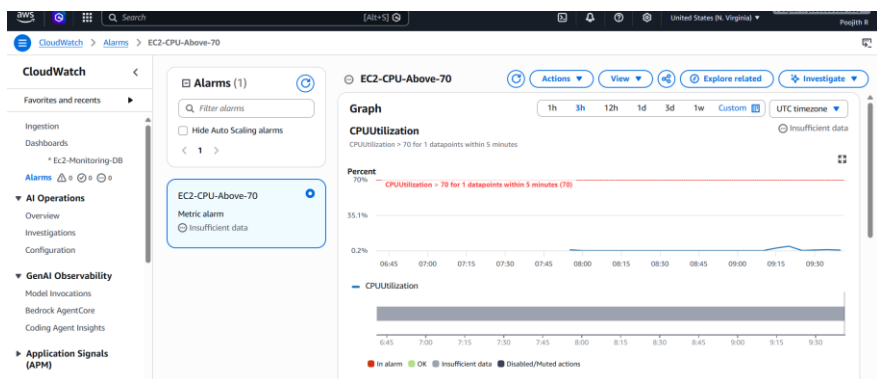
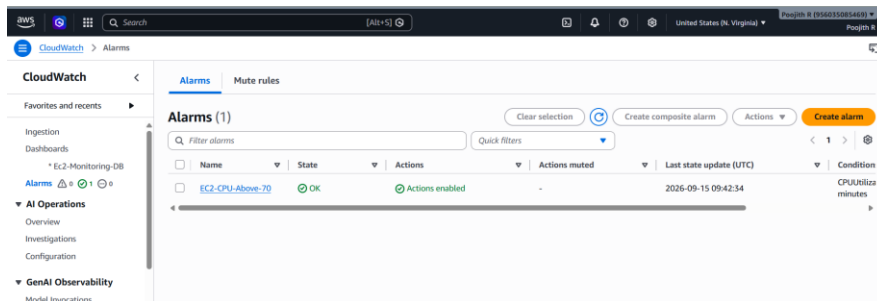
```
cat /var/log/audit/audit.log | grep -E "(start|stop|restart|kill) amazon-cloudwatch-agent"
amazon-cloudwatch-agent.service - Amazon CloudWatch Agent
Loaded: loaded (/etc/systemd/system; vendor preset: disabled)
Active: active (running) since Tue 2026-09-15 09:31:56 UTC; 8s ago
Main PID: 2901 (amazon-cloudwatch-agent)
Tasks: 7 (limit: 1024)
Memory: 52.0M
CPU: 35ms
CGroup: /system.slice/amazon-cloudwatch-agent.service
└─2901 /opt/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -config /opt/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml -envconfig /opt/amazon-cloudwatch-agent/etc/env-c...
```

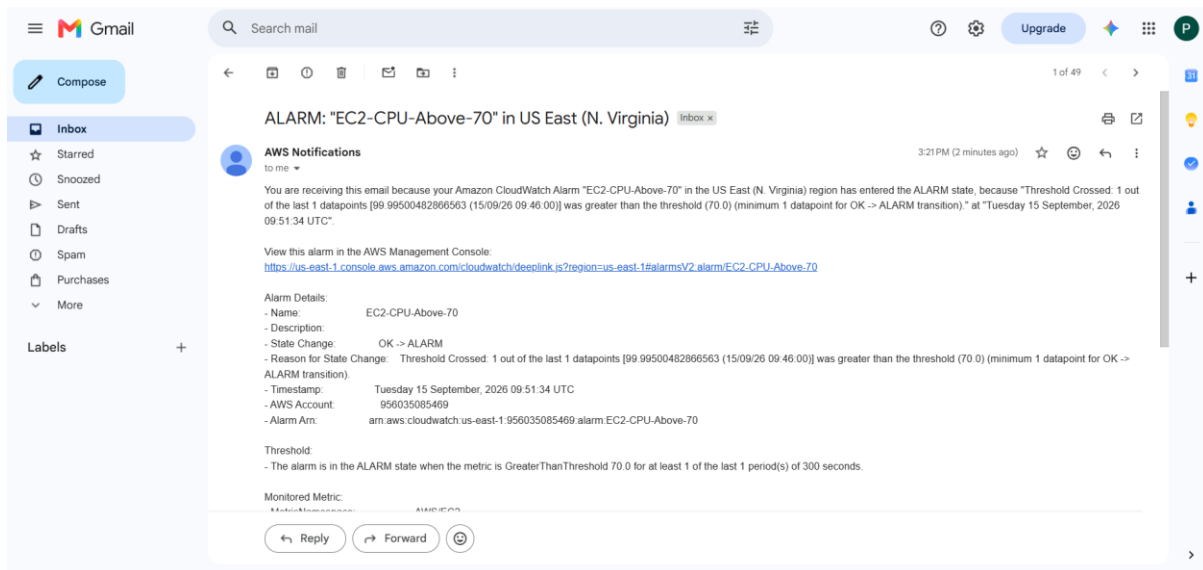


4. Create one alarm to send an alert to email if the CPU utilization is more than 70 percent.

Implementation

1. Opened CloudWatch → Alarms.
2. Selected Create alarm.
3. Selected the required EC2 instance CPU utilization metric.
4. Configured the alarm condition:
 - Metric: CPUUtilization
 - Statistic: Average
 - Threshold: 70%
 - Condition: Greater than 70
5. Configured the evaluation period/datapoints according to the monitoring requirement.
6. Selected the existing SNS topic for notifications.
7. Configured the email subscription through SNS.
8. Created the CloudWatch alarm.
9. Verified the initial alarm state.
10. Monitored the alarm for CPU threshold violations.
11. Verified that SNS notification would be triggered when CPU utilization exceeded the configured threshold.





5. Create a Dashboard and monitor the Tomcat service whether it is running or not and send the alert.

Implementation

1. Logged in to the **EC2 Linux instance**.
2. Verified the Java installation using:

`java -version`

3. Configured the Java environment using `JAVA_HOME`.
4. Downloaded the Apache Tomcat package.
5. Extracted Tomcat under:

`/opt/apache-tomcat-9.0.121`

6. Configured and started the Tomcat service.
7. Verified that Tomcat was listening on port 8080.
8. Created the monitoring script:

`/opt/monitoring.bash`

9. Configured the script to:
 - Retrieve the EC2 Instance ID using **IMDSv2**
 - Check whether Tomcat is running
 - Publish a custom CloudWatch metric

10. Configured the custom metric values:

- 1 → Tomcat running
- 0 → Tomcat stopped

11. Provided the EC2 IAM role with permission to use:

`cloudwatch:PutMetricData`

12. Made the monitoring script executable.

13. Configured **cron** to execute the monitoring script every minute.

14. Verified the custom CloudWatch namespace and metric for Tomcat.

15. Created a CloudWatch alarm for:

tomcat <= 0

16. Configured the alarm to send notifications through the existing SNS topic.

17. Created the **Server-Monitoring** CloudWatch dashboard.

18. Added the Tomcat custom metric to the dashboard.

19. Verified that the dashboard displayed 1 while Tomcat was running.

20. Stopped Tomcat and verified that the metric changed to 0.

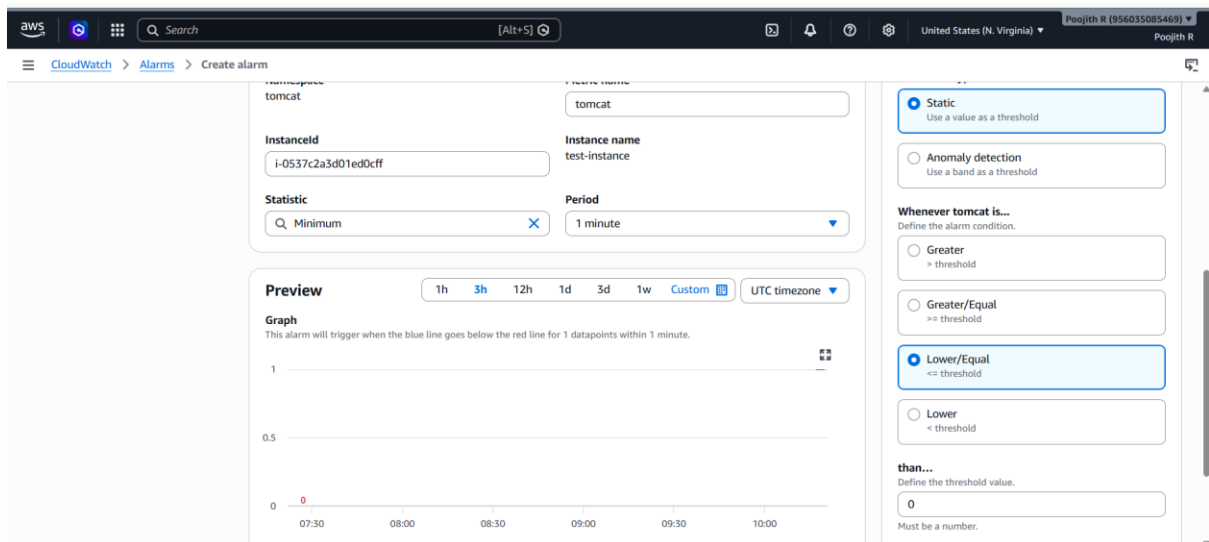
21. Verified that the CloudWatch alarm changed to **ALARM**.

22. Verified the SNS email notification.

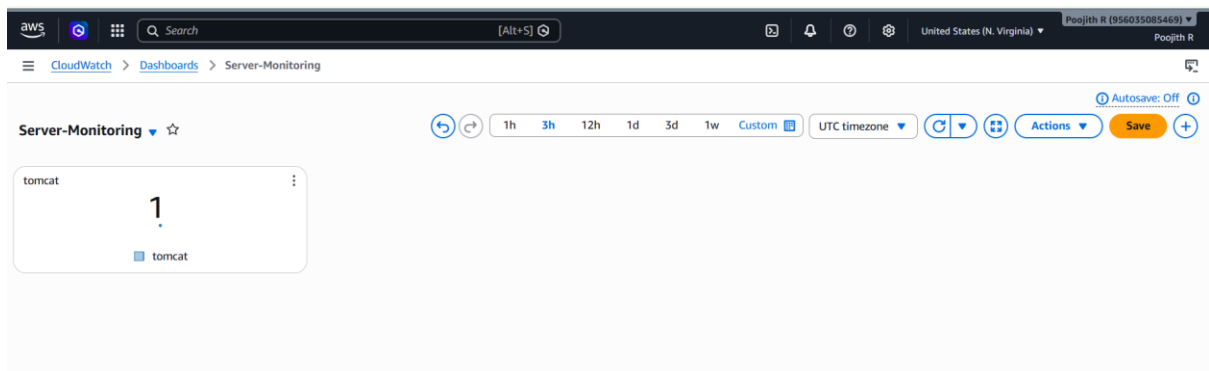
23. Started Tomcat again.

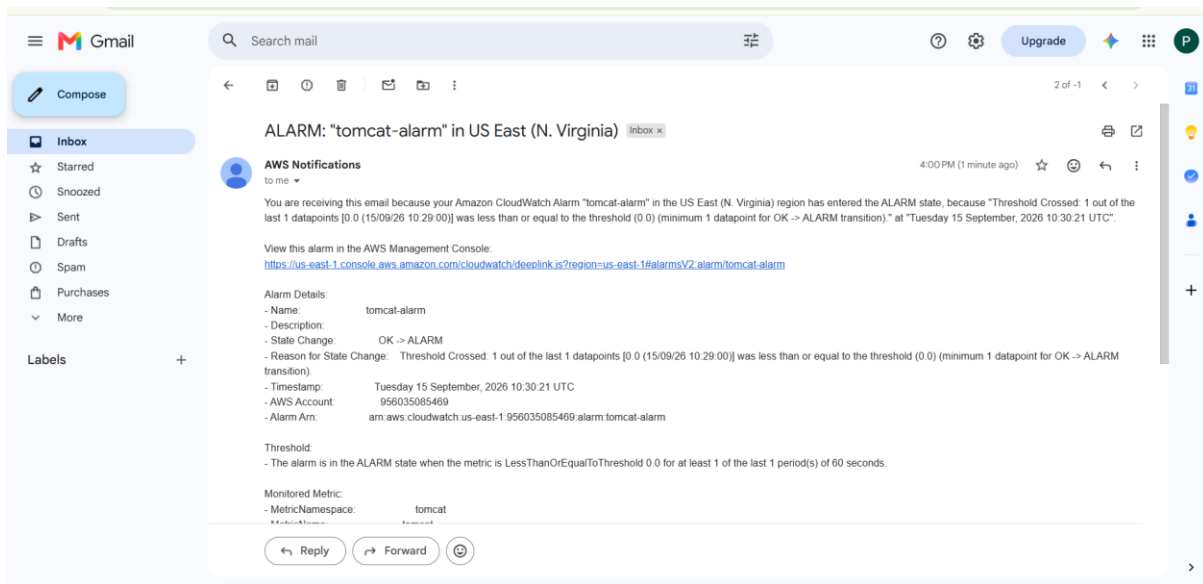
24. Verified that the metric returned to 1.

25. Verified that the alarm returned to **OK**.



```
[root@ip-172-31-36-204 ~]# sudo ss -ltnp | grep 8080
LISTEN 0      100          *:8080          *:8080          users:(("java",pid=26932,fd=45))
[root@ip-172-31-36-204 ~]#
```





6. Create a Dashboard and monitor the Nginx service to send the alert if Nginx is not running.

Implementation

1. Logged in to the EC2 Linux instance.
2. Verified whether Nginx was installed.
3. Installed Nginx when it was not present:

```
sudo dnf install nginx -y
```
4. Started Nginx:

```
sudo systemctl start nginx
```
5. Enabled Nginx to start automatically:

```
sudo systemctl enable nginx
```
6. Verified the Nginx service:

```
sudo systemctl status nginx
```
7. Created the monitoring script:

```
/opt/monitoring-nginx.bash
```
8. Configured the script to:
 - Retrieve the EC2 Instance ID using IMDSv2
 - Check Nginx service status using systemctl
 - Publish Nginx status to CloudWatch
9. Configured the custom metric values:
 - 1 → Nginx running
 - 0 → Nginx stopped
10. Used the EC2 IAM role permission:

cloudwatch:PutMetricData

11. Made the monitoring script executable.
12. Configured cron to execute the script every minute.
13. Verified the custom CloudWatch namespace:

nginx

14. Verified the custom metric:

nginx

15. Created the Nginx-Service-Down-Alarm CloudWatch alarm.
16. Configured the alarm condition:

nginx <= 0

17. Configured the alarm to send notifications to the SNS topic:

Nginx-Service-Down

18. Confirmed the SNS email subscription.
19. Opened the Server-Monitoring CloudWatch dashboard.
20. Added the Nginx custom metric to the dashboard.
21. Saved the dashboard.
22. Verified that the dashboard displayed 1 while Nginx was running.
23. Stopped Nginx:

sudo systemctl stop nginx

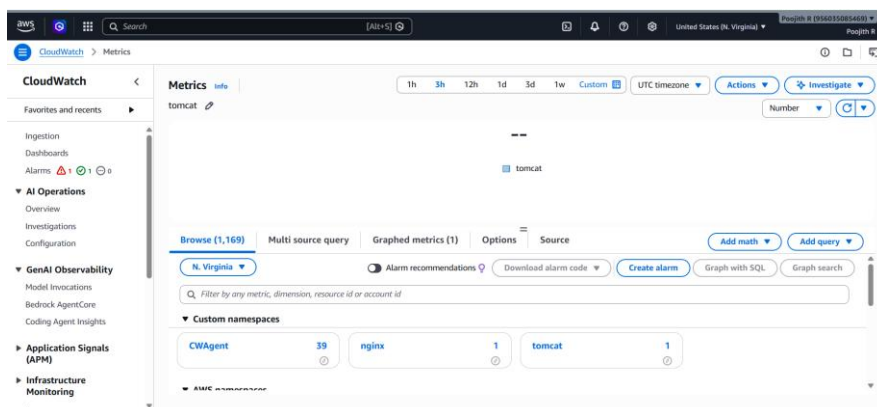
24. Verified that the Nginx metric changed to 0.
25. Verified that the CloudWatch alarm changed to ALARM.
26. Verified receipt of the SNS email alert.
27. Started Nginx again:

sudo systemctl start nginx

28. Verified that Nginx was running.
29. Verified that the metric returned to 1.
30. Verified that the CloudWatch alarm returned to OK.

```
ec2-user@ip-172-31-36-204 ~$ sudo systemctl start nginx
ec2-user@ip-172-31-36-204 ~$ sudo systemctl enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service - /usr/lib/systemd/system/nginx.service.
ec2-user@ip-172-31-36-204 ~$ sudo systemctl status nginx
* nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Tue 2020-09-15 10:35:19 UTC; 15s ago
     Main PID: 30210 (nginx)
       Tasks: 3 (limit: 10559)
      Memory: 3.5M
         CPU: 0ms
      CGroup: /system.slice/nginx.service
              └─30210 "nginx: master process /usr/sbin/nginx"
                  └─30211 "nginx: worker process"
                      └─30212 "nginx: worker process"

Sep 15 10:35:19 ip-172-31-36-204.ec2.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Sep 15 10:35:19 ip-172-31-36-204.ec2.internal nginx[30208]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Sep 15 10:35:19 ip-172-31-36-204.ec2.internal nginx[30208]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Sep 15 10:35:19 ip-172-31-36-204.ec2.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
ec2-user@ip-172-31-36-204 ~$
```



CloudWatch > Alarms > Create alarm

Service: nginx Instance: nginx

Instance ID: i-0537C2a3d01ed0c0ff Instance name: test-instance

Statistic: Minimum Period: 1 minute

Preview: 1h 3h 12h 1d 3d 1w Custom UTC timezone

Graph: This alarm will trigger when the blue line goes below the red line for 1 datapoints within 1 minute.

Threshold: Static Use a value as a threshold

Whenever nginx is... Define the alarm condition.

Greater > threshold

Greater/Equal >= threshold

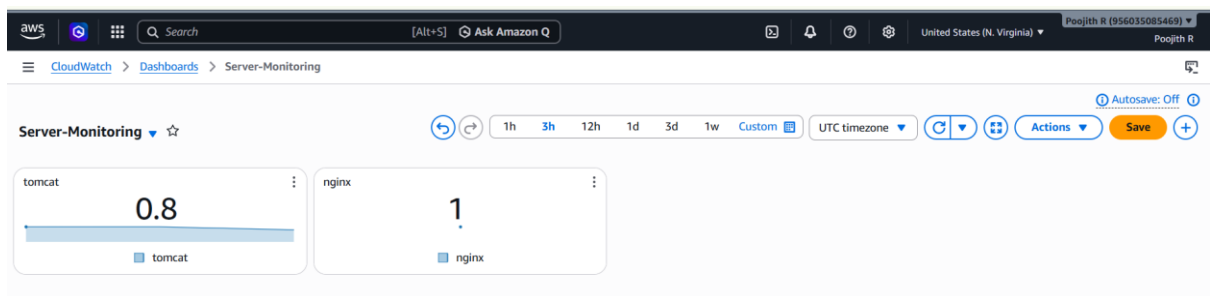
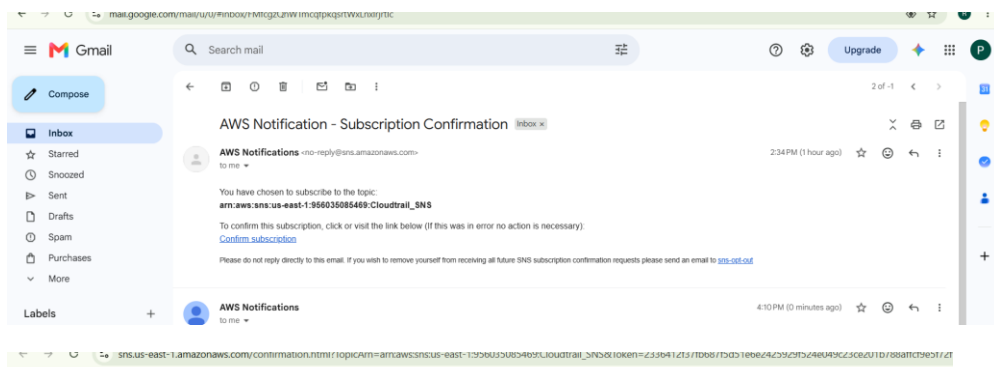
Lower/Equal <= threshold

Lower < threshold

than... Define the threshold value.

1

Must be a number.



```
[ec2-user@ip-172-31-36-204 ~]$ sudo systemctl stop nginx
[ec2-user@ip-172-31-36-204 ~]$ sudo systemctl status nginx
o nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: inactive (dead) since Tue 2026-09-15 10:43:02 UTC; 6s ago
     Duration: 7min 43.196s
    Main PID: 30210 (code=exited, status=0/SUCCESS)
      CPU: 66ms

Sep 15 10:35:19 ip-172-31-36-204.ec2.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Sep 15 10:35:19 ip-172-31-36-204.ec2.internal nginx[30208]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Sep 15 10:35:19 ip-172-31-36-204.ec2.internal nginx[30208]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Sep 15 10:35:19 ip-172-31-36-204.ec2.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
Sep 15 10:43:02 ip-172-31-36-204.ec2.internal systemd[1]: Stopping nginx.service - The nginx HTTP and reverse proxy server...
Sep 15 10:43:02 ip-172-31-36-204.ec2.internal systemd[1]: nginx.service: Deactivated successfully.
Sep 15 10:43:02 ip-172-31-36-204.ec2.internal systemd[1]: Stopped nginx.service - The nginx HTTP and reverse proxy server.
[ec2-user@ip-172-31-36-204 ~]$
```

